

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
import pandas as pd

file_path = "/content/drive/MyDrive/PS_20174392719_1491204439457_log.csv"

df = pd.read_csv(file_path)

print(df.shape)
df.head()
```

(6362620, 11)

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrg	nameDest	oldbalanceDest	newbalanceDest	isFra
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065	0.0	0.0	
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010	21182.0	0.0	
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.0	0.0	

▼ PART 1: CUSTOMER MASTER

First, we will create the `customer_master.csv` file. This involves generating a unique customer for each unique `nameOrig` (originating account) in the PaySim dataset and populating their demographic information, income band, customer segment, risk tier, and other details. We'll use the `Faker` library for realistic data generation and ensure a reproducible random seed.

```
!pip install faker
import pandas as pd
import numpy as np
from faker import Faker
import uuid
import random

# Set a reproducible random seed for Faker and numpy
SEED = 42
Faker.seed(SEED)
np.random.seed(SEED)
random.seed(SEED)

fake = Faker()
```

Requirement already satisfied: faker in /usr/local/lib/python3.12/dist-packages (40.23.0)

▼ Extract Unique Originating Accounts (nameOrig)

The `nameOrig` column in the PaySim dataset represents the originating accounts involved in transactions. We'll treat each unique `nameOrig` as a distinct customer account for the purpose of generating the customer master data.

```
# Get unique nameOrig values from the main DataFrame
unique_name_origs = df['nameOrig'].unique()

print(f"Number of unique originating accounts (customers): {len(unique_name_origs)}")
```

Number of unique originating accounts (customers): 6353307

▼ Generate Customer Master Data

Now, we will iterate through each unique `nameOrig` and generate comprehensive customer profiles including:

- `customer_id`: A unique identifier for the customer.
- `account_id`: The `nameOrig` from the PaySim dataset.
- `full_name`, `age`, `gender`, `occupation`, `income_band`, `customer_segment`, `marital_status`: Realistic demographic details.
- `customer_since`: A realistic date when the customer joined.

- `risk_tier`: Assigned based on a defined distribution (Low, Medium, High).
- `home_city`, `home_state`, `home_country`: Geographic location of the customer.

```
import pandas as pd
import numpy as np
from faker import Faker
import random
import gc
import os

# =====
# CONFIGURATION & PERSONA DEFINITIONS
# =====
SEED = 42
MAX_CUSTOMERS = 100_000
np.random.seed(SEED)
random.seed(SEED)
Faker.seed(SEED)
fake = Faker('en_IN')

PROJECT_FOLDER = "/content/drive/MyDrive/BOB_HACKATHON"
os.makedirs(PROJECT_FOLDER, exist_ok=True)
FILE_PATH = "/content/drive/MyDrive/PS_20174392719_1491204439457_log.csv"

# Enhancement 1 & 3: Persona Mapping and Geographic Integrity
person_profiles = {
    'Student': {'age_range': (18, 24), 'income': 'Low (<5L)', 'segment': 'Retail', 'devices': (3, 5), 'prob': 0.20},
    'Salaried': {'age_range': (25, 55), 'income': 'Medium (5L-15L)', 'segment': 'Premium', 'devices': (1, 3), 'prob': 0.50},
    'Business': {'age_range': (30, 65), 'income': 'High (>15L)', 'segment': 'SME', 'devices': (4, 6), 'prob': 0.20},
    'HNI': {'age_range': (40, 75), 'income': 'High (>15L)', 'segment': 'Corporate', 'devices': (5, 8), 'prob': 0.10}
}

geo_map = {
    "Mumbai": "Maharashtra", "Delhi": "Delhi", "Bangalore": "Karnataka",
    "Hyderabad": "Telangana", "Ahmedabad": "Gujarat", "Chennai": "Tamil Nadu",
    "Kolkata": "West Bengal", "Pune": "Maharashtra", "Jaipur": "Rajasthan", "Lucknow": "Uttar Pradesh"
}

# =====
# GENERATION LOGIC
# =====
print("Loading Account IDs...")
accounts_df = pd.read_csv(FILE_PATH, usecols=["nameOrig"])
account_activity = accounts_df.groupby("nameOrig").size().reset_index(name="txn_count")
customer_accounts = account_activity.sort_values("txn_count", ascending=False).head(MAX_CUSTOMERS)["nameOrig"].tolist()

n_cust = len(customer_accounts)
personas = np.random.choice(list(person_profiles.keys()), n_cust, p=[p['prob'] for p in person_profiles.values()])

data = []
for i, (acc_id, persona) in enumerate(zip(customer_accounts, personas)):
    profile = person_profiles[persona]
    city = np.random.choice(list(geo_map.keys()))
    data.append({
        "account_id": acc_id,
        "customer_id": f"CUST_{i:06d}",
        "persona": persona,
        "age": np.random.randint(*profile['age_range']),
        "income_band": profile['income'],
        "customer_segment": profile['segment'],
        "home_city": city,
        "home_state": geo_map[city],
        "risk_tier": np.random.choice(["Low", "Medium", "High"], p=[0.8, 0.15, 0.05]),
        "full_name": f"{fake.first_name()} {fake.last_name()}",
        "customer_since": (pd.Timestamp("2014-01-01") + pd.to_timedelta(np.random.randint(0, 3650), unit="D"))
    })

customer_master_df = pd.DataFrame(data)
customer_master_df.to_csv(f"{PROJECT_FOLDER}/customer_master.csv", index=False)
print("Persona-Based Customer Master Saved.")
```

```
Loading Account IDs...
Calculating Transaction Frequency...
Generating India-localized Customer Master...
Success! India-localized Customer Master saved to: /content/drive/MyDrive/BOB_HACKATHON/customer_master.csv
```

	account_id	customer_id	age	gender	occupation	income_band	customer_segment	marital_status	risk_tier	home_city
0	C363736674	CUST_000000	56	Female	Environmental health practitioner	Low (<5L)	Premium	Single	Low	Lucknow
1	C724452879	CUST_000001	69	Male	Librarian, public	Medium (5L-15L)	SME	Married	Medium	Ahmedabad
2	C2051359467	CUST_000002	46	Male	Professor Emeritus	Medium (5L-15L)	Retail	Divorced	Low	Mumbai
3	C1976208114	CUST_000003	32	Female	Ambulance person	Low (<5L)	Retail	Divorced	Low	Pune
4	C2098525306	CUST_000004	60	Male	Brewing technologist	Medium (5L-15L)	Retail	Divorced	Medium	Lucknow

Save Customer Master Data

Finally, we'll save the generated `customer_master_df` to a CSV file named `customer_master.csv` in the [/content/drive/MyDrive/](#) directory. This ensures the data is persistently stored and can be used in subsequent parts of the synthetic ecosystem generation.

```
import os

# Create folder inside Google Drive
project_folder = "/content/drive/MyDrive/BOB_HACKATHON"

os.makedirs(project_folder, exist_ok=True)

print(f"Folder ready: {project_folder}")
```

```
Folder ready: /content/drive/MyDrive/BOB_HACKATHON
```

```
output_path = f"{project_folder}/customer_master.csv"

customer_master_df.to_csv(
    output_path,
    index=False
)

print(f"Customer Master saved to: {output_path}")
```

```
Customer Master saved to: /content/drive/MyDrive/BOB_HACKATHON/customer_master.csv
```

PART 3: DEVICE REGISTRY

Now, we will create the `device_registry.csv` file. This dataset will associate each customer with a set of devices, capturing information like device type, operating system, trust score, and usage dates. We aim to simulate realistic device ownership patterns and link devices to `customer_id` from the Customer Master.

```
# Load the customer master to link devices to customers
customer_master_df = pd.read_csv('/content/drive/MyDrive/customer_master.csv')

device_data = []

# Define device types and their distribution
device_types = ['Android', 'iPhone', 'Tablet', 'Laptop', 'Browser']
device_type_distribution = [0.35, 0.30, 0.10, 0.15, 0.10]

# Define OS types (dependent on device type, simplified here)
os_types = {
    'Android': ['Android OS'],
    'iPhone': ['iOS'],
    'Tablet': ['Android OS', 'iOS', 'Windows'],
    'Laptop': ['Windows', 'macOS', 'Linux'],
    'Browser': ['Chrome', 'Firefox', 'Safari', 'Edge'] # Browser itself is the 'OS' here
}

# Device status
device_statuses = ['Active', 'Lost/Stolen', 'Decommissioned']
```

```

device_status_distribution = [0.95, 0.03, 0.02]

# Process customers in chunks to manage memory
CHUNK_SIZE = 50000

for i in range(0, len(customer_master_df), CHUNK_SIZE):
    chunk_customers = customer_master_df.iloc[i:i + CHUNK_SIZE]
    print(f"Processing device chunk {i} to {i + len(chunk_customers)}")

    for _, customer_row in chunk_customers.iterrows():
        customer_id = customer_row['customer_id']

        # Each customer has 1-4 devices
        num_devices = np.random.randint(1, 4)
        customer_devices = [] # To track devices for 'is_primary_device'

        for d in range(num_devices):
            device_id = str(uuid.uuid4())
            device_type = np.random.choice(device_types, p=device_type_distribution)

            # Select OS based on device type (simplified: pick one if multiple options)
            os_type = np.random.choice(os_types.get(device_type, ['Other']))

            device_trust_score = np.random.randint(40, 101) # Range 0-100, more realistic 40-100

            # First seen date: relative to customer_since
            customer_since_date = pd.to_datetime(customer_row['customer_since'])
            first_seen_date = fake.date_between_dates(date_start=customer_since_date, date_end=pd.to_datetime('today'))
            last_seen_date = fake.date_between_dates(date_start=first_seen_date, date_end=pd.to_datetime('today'))

            is_rooted = False # Most devices are not rooted
            if device_type in ['Android', 'iPhone'] and np.random.rand() < 0.02: # Small chance of rooted mobile
                is_rooted = True
            elif device_type == 'Laptop' and np.random.rand() < 0.05: # Small chance of jailbroken/modded laptop
                is_rooted = True

            device_status = np.random.choice(device_statuses, p=device_status_distribution)

            customer_devices.append({
                'device_id': device_id,
                'customer_id': customer_id,
                'device_type': device_type,
                'os_type': os_type,
                'device_trust_score': device_trust_score,
                'first_seen_date': first_seen_date.isoformat(),
                'last_seen_date': last_seen_date.isoformat(),
                'is_primary_device': False, # Will set one to True later
                'is_rooted': is_rooted,
                'device_status': device_status
            })

        # Set one device as primary for the customer if devices exist
        if customer_devices:
            primary_device_idx = np.random.randint(0, len(customer_devices))
            customer_devices[primary_device_idx]['is_primary_device'] = True

        device_data.extend(customer_devices)

# Convert to DataFrame
device_registry_df = pd.DataFrame(device_data)

# Display the first few rows and shape
print("\nDevice Registry DataFrame head:")
display(device_registry_df.head())
print("\nDevice Registry DataFrame shape:")
print(device_registry_df.shape)

```

Processing device chunk 0 to 50000
Processing device chunk 50000 to 100000

Device Registry DataFrame head:

	device_id	customer_id	device_type	os_type	device_trust_score	first_seen_date	last_seen_date	is_primary_device
0	8cf6ec72-fbd8-4b35-880a-57915137ab44	CUST_000000	Tablet	iOS	79	2025-03-10	2025-07-05	False
1	21856102-827e-4bad-96c9-50933c9e3910	CUST_000000	iPhone	iOS	85	2021-03-28	2023-07-13	False
2	941b7869-6cc8-4f28-8657-	CUST_000000	Android	Android OS	99	2023-11-15	2025-09-12	True

Save Device Registry Data

Finally, we'll save the generated `device_registry_df` to a CSV file named `device_registry.csv` in the [/content/drive/MyDrive/](#) directory. This makes the device information available for further analysis and integration into the synthetic ecosystem.

```
import os

PROJECT_DIR = "/content/drive/MyDrive/BOB_HACKATHON"
output_path = f"{PROJECT_DIR}/device_registry.csv"

device_registry_df.to_csv(
    output_path,
    index=False
)

print(f"Device Registry data saved to: {output_path}")
```

Device Registry data saved to: /content/drive/MyDrive/BOB_HACKATHON/device_registry.csv

PART 4: LOCATION REGISTRY

Next, we will create the `location_registry.csv` file. This dataset will store information about the locations associated with each customer, differentiating between their home location and other frequent locations. This helps in understanding customer behavior and identifying potential anomalies.

```
import pandas as pd
import numpy as np
import uuid
import os
from faker import Faker

# Initialize Faker with India locale
fake = Faker('en_IN')
PROJECT_FOLDER = "/content/drive/MyDrive/BOB_HACKATHON"

# Load the India-localized customer master
customer_master_df = pd.read_csv(f"{PROJECT_FOLDER}/customer_master.csv")

location_data = []

# Define common Indian states for fallback
indian_states = ["Maharashtra", "Delhi", "Karnataka", "Telangana", "Gujarat", "Tamil Nadu", "West Bengal", "Rajasthan", "Uttar Pradesh"]
```

```
# Process customers in chunks to manage memory
CHUNK_SIZE = 50000

for i in range(0, len(customer_master_df), CHUNK_SIZE):
    chunk_customers = customer_master_df.iloc[i:i + CHUNK_SIZE]
    print(f"Processing location chunk {i} to {i + len(chunk_customers)}")

    for _, customer_row in chunk_customers.iterrows():
        customer_id = customer_row['customer_id']
        customer_since_date = pd.to_datetime(customer_row['customer_since'])

        # 1. Home Location (Linked to Customer Master demographics)
        home_location_id = str(uuid.uuid4())

        home_first_seen = fake.date_between_dates(date_start=customer_since_date, date_end=pd.to_datetime('today'))
        home_last_seen = fake.date_between_dates(date_start=home_first_seen, date_end=pd.to_datetime('today'))

        location_data.append({
            'location_id': home_location_id,
            'customer_id': customer_id,
            'city': customer_row['home_city'],
            'state': customer_row['home_state'],
            'country': 'India',
            'is_home_location': True,
            'location_trust_score': np.random.randint(85, 101),
            'first_seen': home_first_seen.isoformat(),
            'last_seen': home_last_seen.isoformat()
        })

        # 2. Frequent Locations (0-2 additional Indian locations)
        num_frequent_locations = np.random.randint(0, 3)
        for _ in range(num_frequent_locations):
            freq_location_id = str(uuid.uuid4())

            # Localized generation
            city = fake.city()
            state = np.random.choice(indian_states)

            freq_first_seen = fake.date_between_dates(date_start=customer_since_date, date_end=pd.to_datetime('today'))
            freq_last_seen = fake.date_between_dates(date_start=freq_first_seen, date_end=pd.to_datetime('today'))

            location_data.append({
                'location_id': freq_location_id,
                'customer_id': customer_id,
                'city': city,
                'state': state,
                'country': 'India',
                'is_home_location': False,
                'location_trust_score': np.random.randint(50, 85),
                'first_seen': freq_first_seen.isoformat(),
                'last_seen': freq_last_seen.isoformat()
            })

# Convert to DataFrame
location_registry_df = pd.DataFrame(location_data)

# Save to the dedicated project folder
output_path = f"{PROJECT_FOLDER}/location_registry.csv"
location_registry_df.to_csv(output_path, index=False)

print(f"\nSuccess! India-localized Location Registry saved to: {output_path}")
display(location_registry_df.head())
```

```
Processing location chunk 0 to 50000
Processing location chunk 50000 to 100000

Success! India-localized Location Registry saved to: /content/drive/MyDrive/BOB_HACKATHON/location_registry.csv
```

	location_id	customer_id	city	state	country	is_home_location	location_trust_score	first_seen	last_seen
0	9981e04c-4ab5-4eb9-a681-7792f62a7c3d	CUST_000000	Lucknow	Kerala	India	True	97	2025-06-03	2025-11-03
1	02dfe326-ebad-45b1-9f63-741c14988628	CUST_000000	Dhule	Delhi	India	False	64	2023-01-12	2025-05-10

✓ Save Location Registry Data

Finally, we'll save the generated `location_registry_df` to a CSV file named `location_registry.csv` in the `/content/drive/MyDrive/` directory. This ensures the location information is available for integrating into the synthetic ecosystem.

```
output_path = '/content/drive/MyDrive/location_registry.csv'
location_registry_df.to_csv(output_path, index=False)
print(f"Location Registry data saved to: {output_path}")
```

```
Location Registry data saved to: /content/drive/MyDrive/location_registry.csv
```

✓ PART 2: ACCOUNT MASTER

Next, we will create the `account_master.csv` file. This dataset will contain detailed information for each account, linked to the `customer_id` from the Customer Master. Account characteristics like type, open date, KYC level, and average monthly balance will be generated, with some deriving insights from the original PaySim transaction data.

```
# Load the customer master from the project folder
PROJECT_FOLDER = "/content/drive/MyDrive/BOB_HACKATHON"
customer_master_df = pd.read_csv(f"{PROJECT_FOLDER}/customer_master.csv")

# Prepare data for account master
account_data = []

# Account type distribution
account_types = ['Savings', 'Current', 'Salary', 'Business']
account_type_distribution = [0.4, 0.3, 0.2, 0.1]

# KYC Level distribution
kyc_levels = ['Level 1', 'Level 2', 'Level 3']
kyc_level_distribution = [0.2, 0.5, 0.3]

# Account status distribution
account_statuses = ['Active', 'Dormant', 'Suspended']
account_status_distribution = [0.85, 0.10, 0.05]

# Calculate average balances
print("Calculating average balances from PaySim transactions...")
avg_balance_from_paysim = df.groupby('nameOrig')['newbalanceOrig'].mean().reset_index()
avg_balance_from_paysim.rename(columns={'nameOrig': 'account_id', 'newbalanceOrig': 'paysim_avg_balance'}, inplace=True)

# Merge with customer master
accounts_to_process = customer_master_df[['customer_id', 'account_id', 'customer_since', 'customer_segment']].copy()
accounts_to_process = pd.merge(accounts_to_process, avg_balance_from_paysim, on='account_id', how='left')
accounts_to_process['paysim_avg_balance'] = accounts_to_process['paysim_avg_balance'].fillna(0)

# Process accounts in chunks
CHUNK_SIZE = 50000
today = pd.to_datetime('today')

for i in range(0, len(accounts_to_process), CHUNK_SIZE):
    chunk_accounts = accounts_to_process.iloc[i:i + CHUNK_SIZE]
    print(f"Processing account chunk {i} to {i + len(chunk_accounts)}")

    for _, row in chunk_accounts.iterrows():
        customer_id = row['customer_id']
        account_id = row['account_id']
        customer_since_date = pd.to_datetime(row['customer_since'])

        # Generate account details
        account_type = np.random.choice(account_types, p=account_type_distribution)
        kyc_level = np.random.choice(kyc_levels, p=kyc_level_distribution)
        account_status = np.random.choice(account_statuses, p=account_status_distribution)

        # Open date between customer_since and 5 years prior
        open_date_start_limit = customer_since_date - pd.DateOffset(years=5)
        raw_open_date = fake.date_between_dates(date_start=open_date_start_limit, date_end=customer_since_date)
```

```

# FIXED: Convert date to Timestamp for subtraction
account_open_date = pd.to_datetime(raw_open_date)
account_age_months = int(((today - account_open_date).days) / 30.44)
account_age_months = max(0, account_age_months)

# Balance calculation
paysim_avg_balance = row['paysim_avg_balance']
if paysim_avg_balance > 0:
    avg_monthly_balance = paysim_avg_balance * np.random.uniform(0.8, 1.2)
else:
    avg_monthly_balance = np.random.uniform(50, 1000)

avg_monthly_balance = round(max(0, avg_monthly_balance), 2)

account_data.append({
    'account_id': account_id,
    'customer_id': customer_id,
    'account_type': account_type,
    'account_open_date': account_open_date.strftime('%Y-%m-%d'),
    'account_age_months': account_age_months,
    'kyc_level': kyc_level,
    'account_status': account_status,
    'avg_monthly_balance': avg_monthly_balance,
    'customer_segment': row['customer_segment']
})

# Convert to DataFrame and save
account_master_df = pd.DataFrame(account_data)
output_path = f"{PROJECT_FOLDER}/account_master.csv"
account_master_df.to_csv(output_path, index=False)

print(f"\nSuccess! Account Master saved to: {output_path}")
display(account_master_df.head())

```

Calculating average balances from PaySim transactions...
Processing account chunk 0 to 50000
Processing account chunk 50000 to 100000

Success! Account Master saved to: /content/drive/MyDrive/BOB_HACKATHON/account_master.csv

	account_id	customer_id	account_type	account_open_date	account_age_months	kyc_level	account_status	avg_monthly_balance
0	C363736674	CUST_000000	Savings	2021-09-14	57	Level 2	Active	15
1	C724452879	CUST_000001	Savings	2016-11-03	115	Level 3	Active	
2	C2051359467	CUST_000002	Savings	2016-10-15	115	Level 2	Active	2094
3	C1976208114	CUST_000003	Savings	2015-05-22	132	Level 2	Active	
4	C2098525306	CUST_000004	Salary	2011-07-11	179	Level 2	Active	

Save Account Master Data

Finally, we'll save the generated `account_master_df` to a CSV file named `account_master.csv` in the `/content/drive/MyDrive/` directory, making it available for subsequent steps in building the synthetic banking ecosystem.

```

output_path = '/content/drive/MyDrive/account_master.csv'
account_master_df.to_csv(output_path, index=False)
print(f"Account Master data saved to: {output_path}")

```

Account Master data saved to: /content/drive/MyDrive/account_master.csv

PART 7: IDENTITY EVENT STREAM

This section generates the `identity_events.csv` file, simulating various identity-related actions (e.g., login, password reset) for each customer. Events are linked to customers, devices, and locations, and assigned an event risk based on typical patterns and potential fraud indicators.

```

import pandas as pd
import numpy as np
import uuid

# Enhancement 4 & 5: Identity Event Sequences & Driven Risk
print("Generating Behavioral Identity Event Chains...")

# Logic: Sequences instead of random points
def generate_sequence(cust_id, dev_id, loc_id, start_time):

```

```

seq = []
# Chain: Login -> Registration/Reset (Optional) -> Logout
base_risk = "Low"

# Event 1: Login
seq.append({'event_id': str(uuid.uuid4()), 'customer_id': cust_id, 'timestamp': start_time, 'event_type': 'LOGIN', 'dev

# Event 2: Potential High Risk Action (20% chance)
if np.random.rand() < 0.2:
    start_time += pd.Timedelta(minutes=2)
    action = np.random.choice(['PASSWORD_RESET', 'DEVICE_REGISTRATION', 'MOBILE_CHANGE'])
    seq.append({'event_id': str(uuid.uuid4()), 'customer_id': cust_id, 'timestamp': start_time, 'event_type': action,

# Event 3: Logout
start_time += pd.Timedelta(minutes=5)
seq.append({'event_id': str(uuid.uuid4()), 'customer_id': cust_id, 'timestamp': start_time, 'event_type': 'LOGOUT', 'de
return seq

identity_events_data = []
# Using small sample for code update demonstration
for idx, row in customer_master_df.head(10000).iterrows():
    # Mock device/location lookup for logic update
    events = generate_sequence(row['customer_id'], "DEV MOCK", "LOC MOCK", row['customer_since'] + pd.Timedelta(days=30))
    identity_events_data.extend(events)

identity_events_df = pd.DataFrame(identity_events_data)
identity_events_df.to_csv(f"{PROJECT_FOLDER}/identity_events.csv", index=False)
print("Behavioral Event Chains Saved.")

```

```

Loading datasets...
Customers: 100,000
Devices: 200,214
Locations: 200,011
Building lookup dictionaries...
Lookup dictionaries ready.
Generating identity events...
Processed 10,000/100,000 customers
Processed 20,000/100,000 customers
Processed 30,000/100,000 customers
Processed 40,000/100,000 customers
Processed 50,000/100,000 customers
Processed 60,000/100,000 customers
Processed 70,000/100,000 customers
Processed 80,000/100,000 customers
Processed 90,000/100,000 customers
Processed 100,000/100,000 customers
Creating DataFrame...

=====
SUCCESS
=====
Total Events Generated: 1,200,904
Output File: /content/drive/MyDrive/BOB_HACKATHON/identity_events.csv

```

Save Identity Event Stream Data

Finally, we'll save the generated `identity_events_df` to a CSV file named `identity_events.csv` in the [/content/drive/MyDrive/](#) directory. This event stream is crucial for simulating dynamic identity behaviors and will feed into real-time fraud detection and trust monitoring.

```

PROJECT_FOLDER = "/content/drive/MyDrive/BOB_HACKATHON"
output_path = f"{PROJECT_FOLDER}/identity_events.csv"

identity_events_df.to_csv(output_path, index=False)
print(f"Identity Event Stream data saved to: {output_path}")

```

Identity Event Stream data saved to: /content/drive/MyDrive/BOB_HACKATHON/identity_events.csv

PART 6: DIGITAL TWIN FEATURES

This section generates `digital_twin_features.csv` by calculating various aggregated behavioral features directly from the PaySim transaction history for each account. These features serve as the baseline for the Digital Twin engine, capturing typical transaction patterns and risk indicators.

```

PROJECT_FOLDER = "/content/drive/MyDrive/BOB_HACKATHON"
customer_master_df = pd.read_csv(f"{PROJECT_FOLDER}/customer_master.csv")

print("Calculating India-localized Digital Twin Features...")

```

```

transactions_with_customer = pd.merge(
    df,
    customer_master_df[['customer_id', 'account_id']],
    left_on='nameOrig', right_on='account_id', how='inner'
)

digital_twin_features = transactions_with_customer.groupby('customer_id').agg(
    avg_txn_amount=('amount', 'mean'),
    max_txn_amount=('amount', 'max'),
    total_fraud_txns=('isFraud', 'sum'),
    txn_count=('amount', 'size')
).reset_index()

digital_twin_features['behavior_confidence'] = np.random.randint(60, 100, size=len(digital_twin_features))

output_path = f"{PROJECT_FOLDER}/digital_twin_features.csv"
digital_twin_features.to_csv(output_path, index=False)
print(f"Digital Twin Features saved to: {output_path}")

Calculating India-localized Digital Twin Features...
Digital Twin Features saved to: /content/drive/MyDrive/BOB_HACKATHON/digital_twin_features.csv

```

Save Digital Twin Features Data

Finally, we'll save the generated `digital_twin_features` DataFrame to a CSV file named `digital_twin_features.csv` in the [/content/drive/MyDrive/](#) directory. This comprehensive set of features will be critical for building the Digital Twin baseline.

```

PROJECT_FOLDER = "/content/drive/MyDrive/BOB_HACKATHON"
output_path = f"{PROJECT_FOLDER}/digital_twin_features.csv"

digital_twin_features.to_csv(output_path, index=False)
print(f"Digital Twin Features data saved to: {output_path}")

Digital Twin Features data saved to: /content/drive/MyDrive/BOB_HACKATHON/digital_twin_features.csv

```

PART 5: BENEFICIARY REGISTRY

This section focuses on creating the `beneficiary_registry.csv` by analyzing the transaction graph inherent in the PaySim data. We will identify unique `(nameDest)` for each `(nameOrig)` (customer), calculate transaction counts, relationship age, and assign a beneficiary trust score and type.

```

PROJECT_FOLDER = "/content/drive/MyDrive/BOB_HACKATHON"
customer_master_df = pd.read_csv(f"{PROJECT_FOLDER}/customer_master.csv")

print("Deriving Beneficiary Relationships...")

# Grouping destination accounts
beneficiary_df = df[df['nameOrig'].isin(customer_master_df['account_id'])].groupby(['nameOrig', 'nameDest']).size().reset_index()

# Map back to customer_id
beneficiary_df = pd.merge(beneficiary_df, customer_master_df[['account_id', 'customer_id']], left_on='nameOrig', right_on='account_id')

output_path = f"{PROJECT_FOLDER}/beneficiary_registry.csv"
beneficiary_df.to_csv(output_path, index=False)
print(f"Beneficiary Registry saved to: {output_path}")

Deriving Beneficiary Relationships...
Beneficiary Registry saved to: /content/drive/MyDrive/BOB_HACKATHON/beneficiary_registry.csv

```

Save Beneficiary Registry Data

Finally, we'll save the generated `beneficiary_registry_df` to a CSV file named `beneficiary_registry.csv` in the [/content/drive/MyDrive/](#) directory. This will be a key dataset for building the transaction graph and understanding customer-beneficiary relationships.

```

PROJECT_FOLDER = "/content/drive/MyDrive/BOB_HACKATHON"
output_path = f"{PROJECT_FOLDER}/beneficiary_registry.csv"

# Correcting the variable name from beneficiary_registry_df to beneficiary_df
beneficiary_df.to_csv(output_path, index=False)
print(f"Beneficiary Registry data saved to: {output_path}")

Beneficiary Registry data saved to: /content/drive/MyDrive/BOB_HACKATHON/beneficiary_registry.csv

```

▼ PART 8: CUSTOMER JOURNEYS

This section simulates event sequences (Journeys) by grouping identity events and transactions into logical flows like 'Account Recovery' or 'High Value Transfer'.

```
import pandas as pd
import numpy as np
import uuid

PROJECT_FOLDER = "/content/drive/MyDrive/BOB_HACKATHON"
identity_events_df = pd.read_csv(f"{PROJECT_FOLDER}/identity_events.csv")

print("Generating Customer Journeys...")

journey_types = ['NORMAL_LOGIN', 'BENEFICIARY_ADD', 'PAYMENT', 'ACCOUNT_RECOVERY', 'DEVICE_CHANGE']

# Group events by customer to form journeys
journeys = identity_events_df.groupby('customer_id').agg({
    'timestamp': ['min', 'max'],
    'event_id': 'count'
}).reset_index()

journeys.columns = ['customer_id', 'journey_start', 'journey_end', 'event_count']
journeys['journey_id'] = [f"JRN_{uuid.uuid4().hex[:8]}" for _ in range(len(journeys))]
journeys['journey_type'] = np.random.choice(journey_types, len(journeys))
journeys['journey_risk_score'] = np.random.uniform(0, 100, len(journeys))

output_path = f"{PROJECT_FOLDER}/customer_journeys.csv"
journeys.to_csv(output_path, index=False)
print(f"Journeys saved to: {output_path}")
```

```
Generating Customer Journeys...
Journeys saved to: /content/drive/MyDrive/BOB_HACKATHON/customer_journeys.csv
```

▼ PART 9: TRUST HISTORY

Simulating the evolution of trust scores for each customer over time.

```
# Enhancement 10: Trust History with Real Drivers
print("Generating Explainable Trust History...")

trust_history = []
for cust_id in customer_master_df['customer_id'].head(10000):
    current_trust = 70 # Starting Baseline
    for month in range(1, 7):
        # Simulation of drivers
        # +2 for successful consistent logins, -10 for a password reset simulation
        driver_impact = np.random.choice([2, 5, -10, -5], p=[0.6, 0.2, 0.1, 0.1])
        current_trust = np.clip(current_trust + driver_impact, 0, 100)

        trust_history.append({
            'customer_id': cust_id,
            'date': f"2023-{month:02d}-01",
            'trust_score': current_trust,
            'primary_driver': "Consistent Behavior" if driver_impact > 0 else "Security Event"
        })

trust_history_df = pd.DataFrame(trust_history)
trust_history_df.to_csv(f"{PROJECT_FOLDER}/trust_history.csv", index=False)
print("Explainable Trust History Saved.")
```

```
Generating Trust History...
Trust History saved to: /content/drive/MyDrive/BOB_HACKATHON/trust_history.csv
```

▼ PART 10: NETWORK GRAPH FEATURES

Identifying network-based risks using transaction flows.

```
print("Generating Network Graph Features...")

# Calculate basic network metrics per account
network_features = df.groupby('nameOrig').agg({
    'nameDest': 'nunique',
    'amount': 'count'
})
```

```

}).reset_index()

network_features.columns = ['account_id', 'unique_beneficiaries', 'txn_count']
network_features['network_risk_score'] = np.random.uniform(0, 1, len(network_features)) * network_features['unique_beneficiaries']

# Filter for our active 100k customers
network_features = network_features[network_features['account_id'].isin(customer_master_df['account_id'])]

output_path = f"{PROJECT_FOLDER}/network_features.csv"
network_features.to_csv(output_path, index=False)
print(f"Network Features saved to: {output_path}")
display(network_features.head())

```

Generating Network Graph Features...

Network Features saved to: /content/drive/MyDrive/BOB_HACKATHON/network_features.csv

	account_id	unique_beneficiaries	txn_count	network_risk_score
853	C100025963	2	2	1.179101